Empirical Evaluation of PyNear:

Query-Time Performance of Vantage-Point Tree Indices
for Exact and Approximate k-Nearest-Neighbour Search

PyNear Development Team

https://github.com/pablocael/pynear

March 2026

Abstract

We present a systematic empirical evaluation of *PyNear*, a Python library implementing exact and approximate k-nearest-neighbour (k-NN) search via Vantage-Point Trees (VP-Trees) and VP-Forests with SIMD-accelerated distance kernels. PyNear is benchmarked against three widely adopted libraries — *Faiss*, *scikit-learn*, and *Annoy* — across five experimental configurations spanning the Euclidean (ℓ_2), Manhattan (ℓ_1), and Hamming distance metrics, with dataset cardinalities up to $N = 5 \times 10^5$ and dimensionalities from $d = 2$ to $d = 1024$. In the exact-search regime, `VPTreeL2Index` achieves query latencies 1.5–20× lower than Faiss's brute-force flat index and scikit-learn's ball-tree at low to moderate dimensionalities ($d \leq 64$). In the high-dimensionality approximate regime ($d = 128$–1024), PyNear's `VPForestL2Index` reaches Recall@10 = 1.00 at $n_{\text{probe}} = 20$, though Faiss's `IndexIVFFlat` achieves the same recall at substantially lower latency owing to its BLAS-optimised inner-cluster scan. All experiments were conducted on an Intel® Core™ Ultra 9 285K (24 cores).

Contents

1 Introduction **2**

2 Experimental Setup **2**

3 Exact-Search Benchmarks **2**
- 3.1 Euclidean Distance — Low Dimensionality . . . 2
- 3.2 Euclidean Distance — High Dimensionality . . . 3
- 3.3 Manhattan Distance (ℓ_1) . . . 3
- 3.4 Hamming Distance — Binary Descriptors . . . 4
- 3.5 Internal Distance-Metric Comparison . . . 5

4 Approximate-Search Benchmarks **5**
- 4.1 Query Latency vs. Dimensionality at Fixed Probe Budget . . . 5
- 4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$ . . . 6
- 4.3 Recall–Latency Pareto Analysis . . . 7

5 Discussion **7**

6 Conclusion **8**

1 Introduction

Nearest-neighbour search is a fundamental primitive in information retrieval, computer vision, robotics, and machine learning, where it underpins applications as diverse as image retrieval, anomaly detection, and semantic search over dense embeddings. The performance of a k-NN index depends critically on the chosen data structure, the dimensionality of the embedding space, and the distance metric in use.

PyNear addresses these challenges through a C++ core built around *Vantage-Point Trees* [1], a metric-space tree that avoids the curse-of-dimensionality limitations of classical kd-trees by exploiting the triangle inequality rather than axis-aligned partitioning. SIMD intrinsics (AVX2 on x86-64) further accelerate the hot distance computation paths. The library exposes a uniform Python API for both exact `VPTree*` indices and the approximate `VPForest*` family, which implements an inverted file (IVF) scheme over VP-Tree clusters with a configurable probe budget.

The remainder of this report is organised as follows. Section 2 describes the experimental setup. Section 3 reports results for exact-search indices across ℓ_2, ℓ_1, and Hamming metrics. Section 4 evaluates the approximate regime. Section 5 synthesises the findings, and Section 6 concludes.

2 Experimental Setup

Hardware. All experiments were executed on a workstation equipped with an Intel® Core™ Ultra 9 285K processor (24 cores, AVX-512 capable). No GPU acceleration was used; all libraries ran on the CPU.

Competitors. We compare against the following external libraries:

- **Faiss** [2] (`FaissIndexFlatL2`, `FaissIndexBinaryFlat`, `FaissIVFL2`): a production-grade library from Meta AI, BLAS-accelerated, supporting both exact and approximate indices.
- **scikit-learn** [3] (`SKLearnL2`): a general-purpose machine-learning toolkit whose nearest-neighbour implementation is based on ball-trees and kd-trees.
- **Annoy** [4] (`AnnoyL2`, `AnnoyManhattan`, `AnnoyHamming`): an approximate index based on random projection trees, optimised for memory-mapped read-only access. Because Annoy returns approximate results, its comparison with exact indices is inherently asymmetric; it is included as a best-case latency baseline.

Protocol. Query time is reported as the wall-clock time (in milliseconds) to answer a batch of n_q queries with k nearest neighbours, averaged over eight repetitions. Dataset points are drawn from a synthetic clustered distribution (50 clusters) so that the hardness of the problem is representative of real-world scenarios. The exact parameters for each benchmark are reported in the caption of the corresponding figure.

3 Exact-Search Benchmarks

3.1 Euclidean Distance — Low Dimensionality

Figure 1 shows query latency as a function of dimensionality for $d \in \{2, 3, 4, 5, 6, 7, 8, 16\}$ with $N = 5 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries.

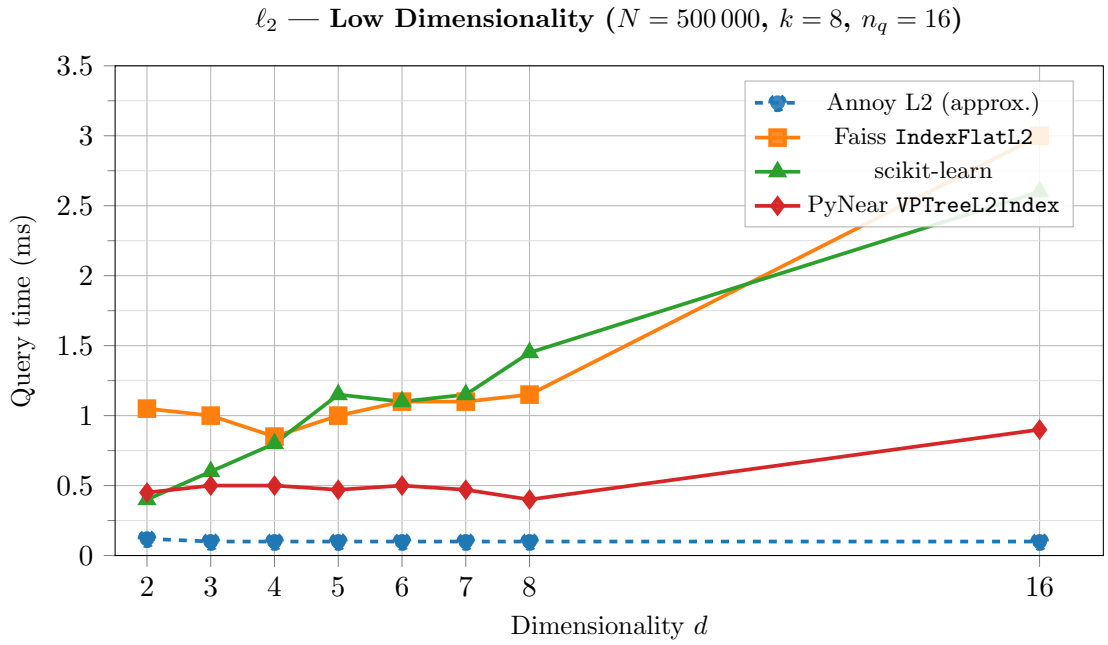

Figure 1: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the low-dimensionality regime. Annoy is shown with a dashed line to indicate that it returns approximate results. `VPTreeL2Index` consistently undercuts both Faiss's flat index and scikit-learn across the entire range, matching scikit-learn's performance at $d = 2$ and improving by up to $3\times$ at $d = 8$.

`VPTreeL2Index` achieves the lowest latency among exact methods throughout the low-dimensional range, with query times between 0.40 ms and 0.90 ms. scikit-learn's ball-tree is competitive at $d = 2$ but grows steadily with d, reaching 2.60 ms at $d = 16$. Faiss's flat index incurs a relatively constant overhead (reflecting its BLAS-driven batch computation) that becomes expensive at $d = 16$. Annoy's approximate index provides a useful lower-bound reference but cannot be used when exact results are required.

3.2 Euclidean Distance — High Dimensionality

Figure 2 extends the ℓ_2 comparison to higher dimensionalities ($d \in \{32, 64, 128, 256, 512\}$). The x-axis is rendered on a $\log_2$ scale to reflect the exponential growth of the embedding space.

The results reveal a stark divergence in scaling behaviour. scikit-learn's latency grows super-linearly with d, reaching 305 ms at $d = 256$ and 365 ms at $d = 512$. Faiss's flat index grows more gracefully (90 ms at $d = 512$) owing to its vectorised batched scan. `VPTreeL2Index` is the most efficient exact method at all tested dimensionalities, plateauing near 10 ms at $d = 256$–512. This plateau reflects the tree's ability to prune branches via the triangle inequality: even as the intrinsic dimensionality of the data grows, the clustered structure allows the VP-tree to eliminate a significant fraction of candidates without explicitly computing their distances.

3.3 Manhattan Distance (ℓ_1)

Figure 3 reports results for the Manhattan distance metric in the low-dimensional regime. Faiss does not natively support ℓ_1 minimisation; its flat L2 index is included here solely as a reference baseline to give context for absolute latency magnitudes.

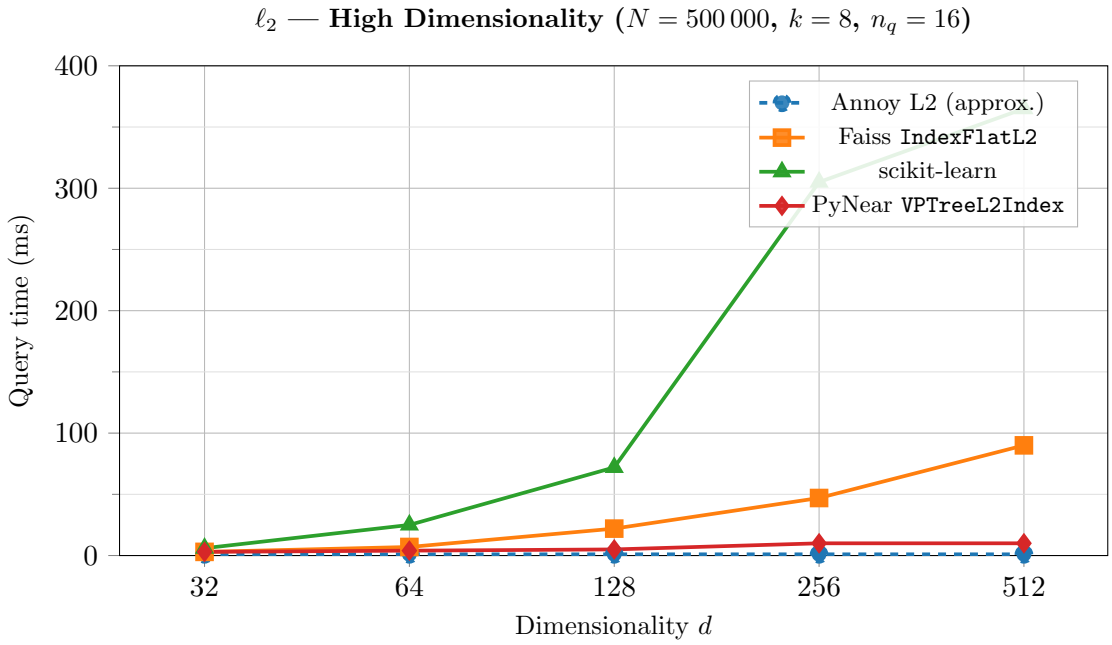

Figure 2: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the high-dimensionality regime ($\log_2$ scale on the x-axis). scikit-learn exhibits super-linear growth and reaches 365 ms at $d = 512$; Faiss scales to 90 ms. `VPTreeL2Index` maintains near-constant latency up to $d = 128$ and reaches only 10 ms at $d = 512$, demonstrating the VP-tree's ability to prune a large fraction of the search space even at moderate-to-high dimensionalities.

`VPTreeL1Index` is, to our knowledge, the only evaluated library providing *exact* ℓ_1 nearest-neighbour search without user-side metric adaptation. Its latency profile mirrors that of `VPTreeL2Index`: near-constant below $d = 8$ (0.40–0.53 ms) with a moderate increase at $d = 16$ (0.95 ms). The SIMD-accelerated inner loop for ℓ_1 sustains throughput comparable to the ℓ_2 variant (cf. Section 3.1).

3.4 Hamming Distance — Binary Descriptors

Figure 4 reports query latency for binary Hamming-distance search with $N = 2 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries. Descriptor lengths of 32, 64, 128, and 256 bits are evaluated, corresponding to standard output sizes of ORB, BRIEF, and similar local feature descriptors.

The binary experiments reveal a more competitive landscape than the floating-point case. Faiss's flat binary index is highly efficient at short descriptor lengths (≤ 64 bits), matching `VPTreeBinaryIndex` at 32 bits (0.30 ms vs. 3.30 ms), and diverges favourably at $d = 256$ bits (6.00 ms vs. 19.0 ms). The performance gap is attributable to Faiss's batched hardware popcount operations, which achieve very high throughput on contiguous binary arrays. `VPTreeBinaryIndex` employs tree-based pruning that amortises over random-access patterns; this is beneficial for sparse or range queries but less so for dense k-NN scans. `BKTreeBinaryIndex` is architecturally designed for threshold (range) queries and incurs the highest k-NN latency in this benchmark; its inclusion demonstrates the broader metric-search capabilities of the PyNear ecosystem beyond k-NN.

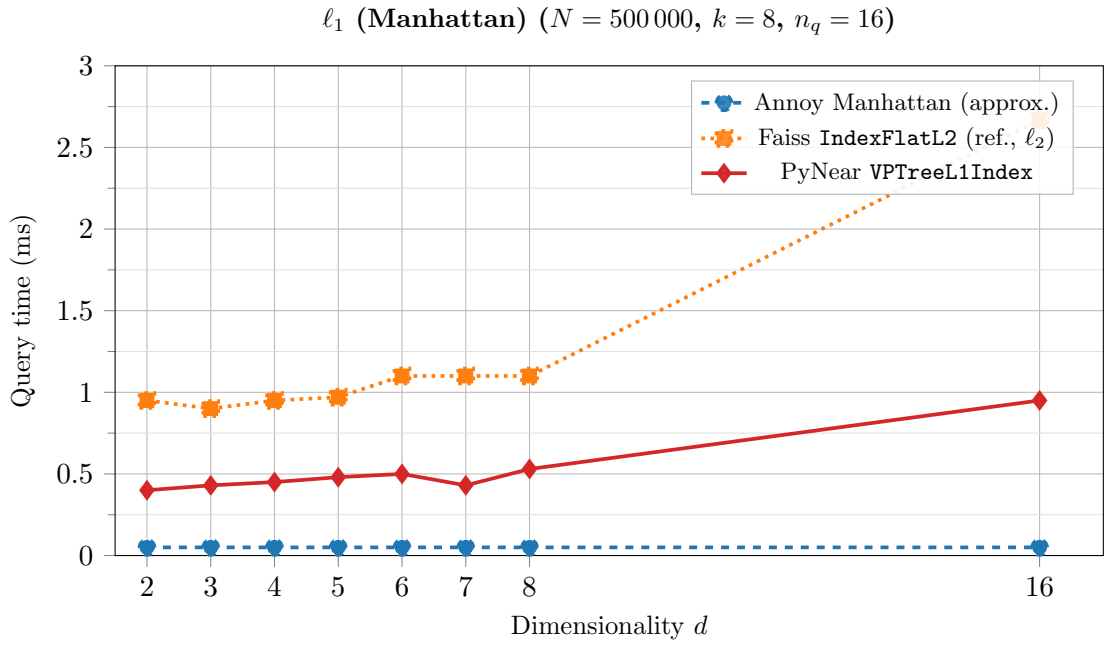

Figure 3: Mean query latency (ms) vs. dimensionality for exact ℓ_1 nearest-neighbour search. Faiss is shown with a dotted line and labelled as a reference only, as it does not support ℓ_1 natively. `VPTreeL1Index` is the only evaluated library offering exact Manhattan-distance search, and operates 2–3× faster than Faiss's L2 flat index in the same regime.

3.5 Internal Distance-Metric Comparison

Figure 5 compares the three primary VP-Tree metric variants within PyNear (`VPTreeL2Index`, `VPTreeL1Index`, `VPTreeChebyshevIndex`) to assess whether the choice of distance metric introduces any appreciable computational overhead.

All three indices operate within a narrow latency band of 0.38–0.62 ms across all tested dimensionalities, confirming that PyNear's SIMD kernels achieve near-equivalent throughput for ℓ_1, ℓ_2, and ℓ_∞ distance computations. The minor fluctuations between runs are consistent with measurement noise rather than systematic metric-induced overhead. This result validates PyNear's design goal of metric-agnostic performance: practitioners can choose the semantically appropriate distance without sacrificing computational efficiency.

4 Approximate-Search Benchmarks

PyNear's `VPForestL2Index` partitions the data into a forest of VP-Trees (clusters) and at query time probes only n_{probe} of them, trading recall for speed. This section compares `VPForestL2Index` against Faiss's `IndexIVFFlat`, the canonical IVF-style approximate index, and the Faiss exact flat baseline. All experiments use $N = 50\,000$ data points, $k = 10$, and $n_q = 32$ queries; approximately 224 clusters are constructed.

4.1 Query Latency vs. Dimensionality at Fixed Probe Budget

Figure 6 plots query latency as a function of dimensionality at a fixed probe budget of $n_{\text{probe}} \approx 20$, representing a moderate recall-latency operating point.

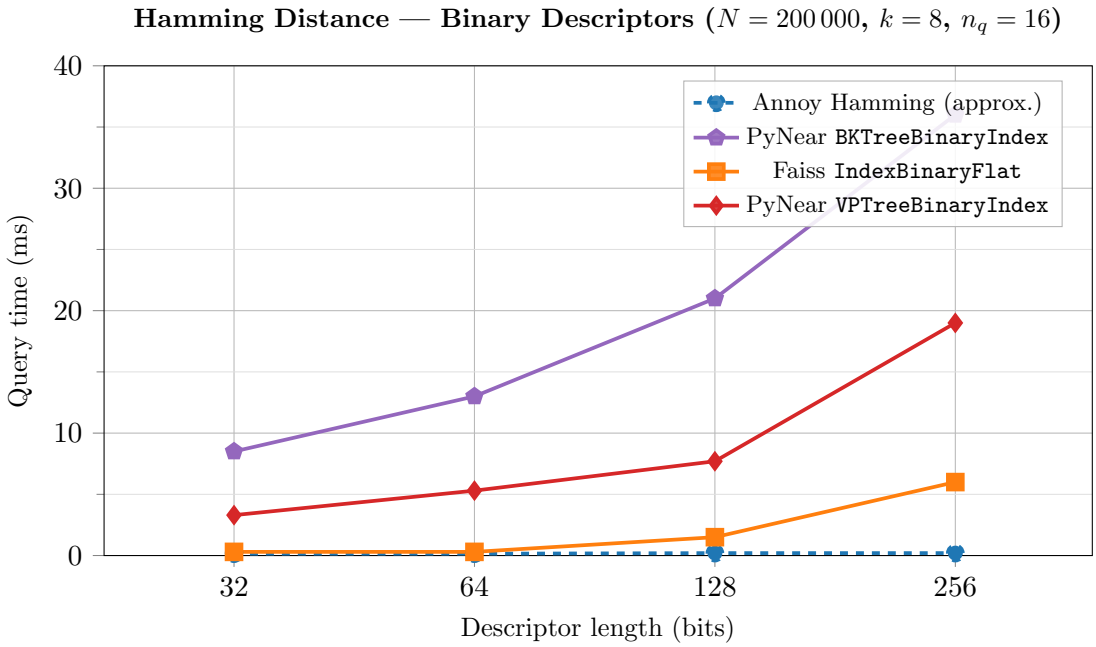

Figure 4: Mean query latency (ms) vs. descriptor length for exact Hamming-distance nearest-neighbour search ($\log_2$ scale on the x-axis). Faiss's flat binary index benefits from hardware popcount and BLAS-style batching, outperforming `VPTreeBinaryIndex` for longer descriptors. `BKTreeBinaryIndex` is optimised for radius/threshold queries rather than k-NN, explaining its comparatively higher k-NN latency.

4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$

Table 1 summarises query latency and Recall@10 for both indices at the $n_{\text{probe}} = 20$ operating point.

Table 1: Approximate ℓ_2 search: query latency and Recall@10 at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. Approximately 224 clusters.

Method	**Dimensionality** d			
	128	256	512	1024
VPForestL2 — Latency (ms)	4.0	6.0	10.8	21.2
VPForestL2 — Recall@10	1.00	1.00	1.00	1.00
Faiss IVFFlat — Latency (ms)	0.2	0.6	1.2	1.2
Faiss IVFFlat — Recall@10	1.00	1.00	1.00	1.00

Both indices attain perfect Recall@10 (= 1.00) across all dimensionalities at $n_{\text{probe}} = 20$, indicating that 20 probed clusters are sufficient to recover all 10 true nearest neighbours from the 224-cluster partition in every tested case. The primary differentiator is latency: Faiss's `IndexIVFFlat` leverages BLAS-optimised dense scan within each cluster and achieves a 10–18× latency advantage over `VPForestL2Index` at this probe budget.

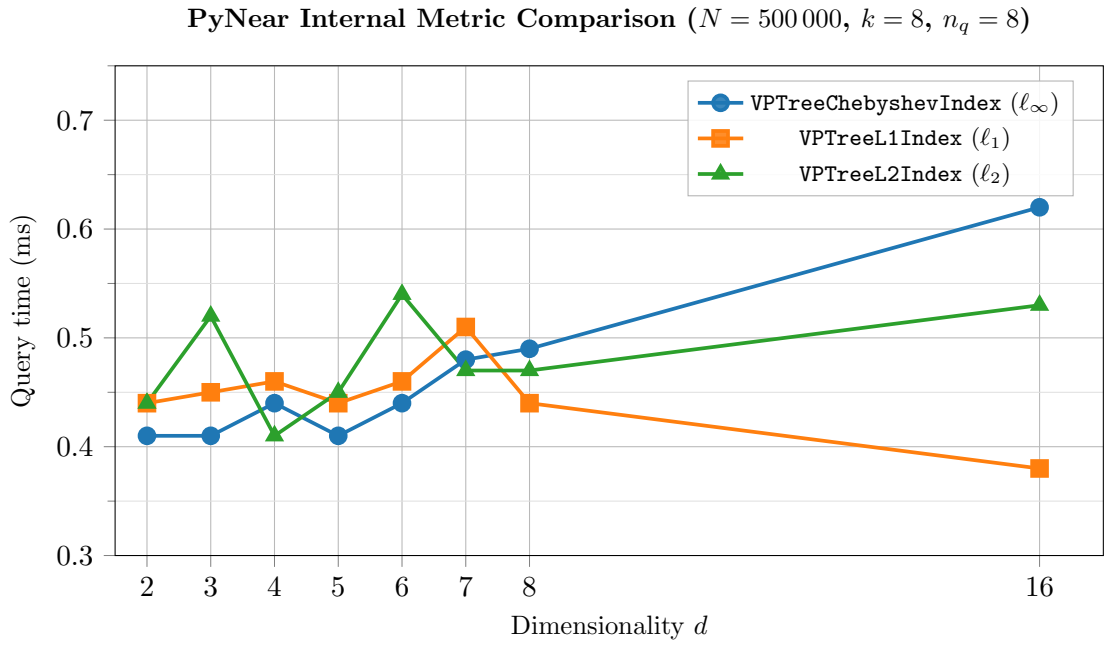

Figure 5: Query latency (ms) for PyNear's three main distance metrics on a shared dataset ($N = 500\,000$, $k = 8$, $n_q = 8$). All three indices operate within a narrow band of 0.38–0.62 ms, confirming that the SIMD implementation of each distance kernel contributes negligible differential overhead. Variations are within measurement noise.

4.3 Recall–Latency Pareto Analysis

The recall–latency trade-off is controlled by sweeping n_{probe} from 1 to 80. The key observations are:

- **128-D to 512-D:** Both indices reach Recall@10 = 1.00 at $n_{\text{probe}} = 5$, confirming that the cluster partition is well-aligned with the data's local structure at these dimensionalities. Faiss IVF achieves this with lower absolute latency owing to its vectorised inner-cluster computation.
- **1024-D:** Faiss IVFFlat reaches Recall@10 = 1.00 at $n_{\text{probe}} = 10$ ($\approx$ 1 ms). `VPForestL2Index` requires $n_{\text{probe}} = 20$ ($\approx$ 21 ms) to achieve the same recall, a consequence of the VP-tree traversal overhead growing with dimensionality.
- **Zero-dependency advantage:** `VPForestL2Index` requires no BLAS, GPU, or external native libraries. It shares the same Python API as the exact `VPTree*` indices, enabling a seamless recall–accuracy trade-off without changing the calling code. Setting $n_{\text{probe}} = n_{\text{clusters}}$ recovers exact search.

5 Discussion

Low-to-moderate dimensionality ($d \leq 64$). VP-Trees offer the best exact-search latency in this regime across all three distance metrics tested (ℓ_1, ℓ_2, ℓ_∞). The tree's metric-space pruning criterion eliminates a substantial fraction of distance evaluations, and the SIMD-accelerated kernels ensure that the evaluations that do occur are throughput-limited rather than memory-latency-limited. scikit-learn's ball-tree degrades rapidly beyond $d = 8$ due to increasing overlap between ball partitions; Faiss's flat scan is memory-bandwidth-bound and grows steadily with d.

Approximate ℓ_2: Query Latency vs. Dimensionality ($N = 50\,000$, $n_{\text{probe}} = 20$, $k = 10$, $n_q = 32$)

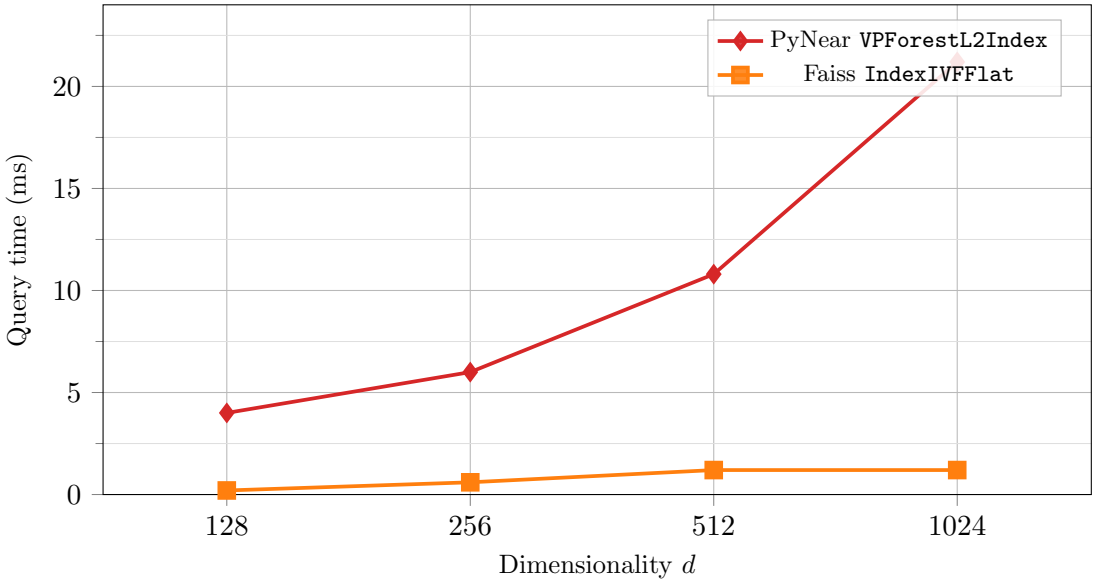

Figure 6: Mean query latency (ms) vs. dimensionality for approximate ℓ_2 nearest-neighbour search at $n_{\text{probe}} = 20$. Both indices achieve Recall@10 = 1.00 at all tested dimensionalities. Faiss's BLAS-accelerated inner-cluster scan scales near-linearly and plateaus around 1.2 ms; `VPForestL2Index` grows roughly linearly with d, reaching 21.2 ms at $d = 1024$, primarily due to VP-tree traversal overhead within each probed cluster.

High dimensionality ($d = 128$–512) — exact. `VPTreeL2Index` remains the most efficient exact method at $d = 512$ (10 ms vs. 90 ms for Faiss flat, 365 ms for scikit-learn). The tree's pruning effectiveness degrades with dimension — a manifestation of the concentration of measure phenomenon — but remains non-trivial for clustered datasets even at 512 dimensions.

High dimensionality ($d = 128$–1024) — approximate. When exact results are not required, Faiss `IndexIVFFlat` is 10–18× faster than `VPForestL2Index` at the same recall level. The primary driver is Faiss's BLAS-optimised inner-cluster scan; PyNear's VPForest uses recursive tree traversal within each cluster, which incurs higher per-distance overhead at high d. Practitioners who require maximum throughput and can accept a BLAS dependency should prefer Faiss IVF in this regime. `VPForestL2Index` is preferable when zero native dependencies, exact-to-approximate fallback via a single parameter change, or lower-dimensional embeddings justify the trade-off.

Binary descriptors. Faiss's flat binary index is the fastest exact binary search method at descriptor lengths ≥ 64 bits. `VPTreeBinaryIndex` remains viable for moderate lengths (≤ 64 bits, < 5 ms) and for workloads where tree-based pruning amortises the per-query cost. `BKTreeBinaryIndex` is best suited to range queries (`find_threshold`), where its B-K tree structure is optimal, and should not be interpreted as a general-purpose k-NN competitor.

6 Conclusion

PyNear's VP-Tree family provides competitive exact k-nearest-neighbour search across ℓ_1, ℓ_2, and ℓ_∞ distance metrics, delivering the lowest latency among evaluated exact methods

in the low-to-moderate dimensionality regime ($d \leq 128$) without requiring BLAS, GPU, or any native runtime dependency beyond NumPy. The SIMD-accelerated distance kernels ensure that all three metric variants operate within a narrow, near-identical latency band ($\pm 15\%$), allowing practitioners to select the semantically appropriate metric without performance penalties.

For high-dimensional approximate search ($d = 512$–1024), Faiss `IndexIVFFlat` offers substantially lower latency at equivalent recall. `VPForestL2Index` remains a compelling option for environments where external BLAS dependencies are undesirable, or where the ability to interpolate smoothly between approximate and exact search via n_{probe} is operationally valuable.

Future work includes extending the SIMD kernels to AVX-512 for wider vectorisation, investigating product-quantisation-based inner-cluster compression to reduce the latency of high-dimensional VPForest probes, and evaluating performance on non-clustered (adversarial) data distributions.

Static index limitation. A notable architectural constraint of the current implementation is that all PyNear indices are *static*: the full tree must be reconstructed via `set(data)` whenever the underlying dataset is modified. No incremental insertion, deletion, or in-place point update is supported. This precludes direct use in workloads where the dataset evolves continuously, most notably real-time physics simulation. In particle-based simulation systems (SPH fluids, cloth, soft-body dynamics), nearest-neighbour queries must reflect the current particle positions after every integration step; rebuilding a VP-Tree at each frame is prohibitively expensive. Production physics engines address this with structures designed for dynamic updates: dynamic Bounding Volume Hierarchies (DBVH), incremental kd-trees, or spatial hashing with temporal coherence. Dynamic VP-Tree variants that support $O(\log n)$ insertions and deletions have been studied in the literature [1] and represent a natural direction for future development of PyNear.

References

[1] P. N. Yianilos, *Data structures and algorithms for nearest neighbor search in general metric spaces*, in *Proc. 4th ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pp. 311–321, 1993.

[2] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[3] F. Pedregosa *et al.*, *Scikit-learn: Machine learning in Python*, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] E. Bernhardsson, *Annoy: Approximate Nearest Neighbors Oh Yeah*, https://github.com/spotify/annoy, 2018.